

MINI PROJECT REPORT

Dave: Reissue — A Modern Remake of Dangerous Dave

Subject: User Interface Design

Submitted by

Name	USN
Ujwal Hegde	NNM23IS201
K Aravind Kamath	NNM23IS082

1. Abstract

Dave: Reissue is a browser-based remake of the 1988 disk operating system game Dangerous Dave, developed as a mini project for the subject User Interface Design. The remake preserves the objective of the original, in which the player collects a key and reaches the exit door of each level, while rebuilding the interface and the controls around present-day expectations. Ten levels, three playable characters, a heads-up display, a pause screen and a persistent save are implemented entirely on the client using Next.js, TypeScript and the HTML5 canvas element. The work is organised around nine documented usability limitations of the original game, each of which is addressed by a specific feature of the remake.

2. Introduction

Dangerous Dave, released in 1988, is a side-scrolling platform game in which the player guides the title character through a series of single-screen levels, collecting a trophy and a key before reaching the door that leads to the next level. The game was written for early personal computers and reflects the constraints of that hardware. The display scrolled a full screen at a time, the artwork was rendered as coarse pixel tiles, contact with any hazard was immediately fatal, the controls were fixed, and no progress was retained once the program was closed. These characteristics are now understood as usability defects rather than stylistic choices.

This project is a modern remake of that game, built to run in a web browser. Its purpose is not to reproduce the original faithfully but to modernise its interface and to correct the usability flaws that the original exhibits, while retaining the structure and the pace that made it recognisable. The application is client-side only: there is no server component, no backend and no external game engine, and the entire world is drawn procedurally into a single canvas element. Every interaction, from the main menu to the level complete screen, has been designed as part of the same visual system.

3. Objectives

- To study the interface and interaction design of the original Dangerous Dave and to identify its principal usability limitations.
- To design and implement a browser-based remake of the game using Next.js, TypeScript and the HTML5 canvas element.

- To resolve each identified limitation through a specific and demonstrable feature of the remake.
- To provide a consistent visual language in which colour, motion and feedback communicate the state of the game unambiguously.
- To implement frame-rate independent physics so that the behaviour of the game does not depend on the speed of the machine.
- To retain player progress across sessions through client-side storage, without requiring an account or a server.

4. Tools and Technologies Used

The technologies used in the development of the project are listed below.

Technology	Purpose
Next.js (App Router)	Application shell, routing and the production build.
TypeScript	Static typing across the game engine, level data and interface components.
HTML5 Canvas	Immediate-mode drawing surface on which the whole game world is rendered.
Tailwind CSS	Utility-first styling of the menus, heads-up display and overlays.
localStorage	Client-side persistence of progress, chosen character, gems and checkpoints.
Git and GitHub	Version control and hosting of the source repository.
Vercel	Deployment of the production build as a static, client-side application.

5. Key Features

- Ten hand-authored levels, each defined as a grid of tile characters sixteen rows in height.
- The complete game world is drawn procedurally into one canvas element, with no sprite or image assets.
- A fixed simulation timestep with interpolated rendering, giving identical behaviour at any refresh rate.

- Three playable characters, each with its own speed, jump height and number of hearts, selectable at any time.
- A three-heart health system with knockback and a brief period of invulnerability after each hit.
- Checkpoint tiles that record the player position and restore it after a death.
- A gun, introduced in the sixth level, with capped ammunition that regenerates over time so a level cannot become unwinnable.
- A jetpack, introduced in the seventh level, whose fuel recharges on the ground and from canisters placed in the level.
- Patrolling enemies, spikes, wall-mounted turrets and breakable timber crates as hazards and obstacles.
- Tutorial hints that appear as the player walks into the region of the level to which they apply.
- A heads-up display showing hearts, gems, ammunition, jetpack fuel and elapsed time, with pause and level complete overlays.
- Persistent progress, character selection and checkpoint data stored in the browser and validated when read.

6. Pitfalls in the Original Game and Their Resolution

The design of the remake was driven by nine limitations observed in the original game. Each limitation and the corresponding solution implemented in this project are given in the table below.

Sl. No.	Limitation in the Original Game	Solution Implemented
1	Camera. The original scrolled in whole screen widths, so the view snapped forward and the player briefly lost sight of the surroundings.	The camera follows the player through an exponential interpolation with a look-ahead offset and is clamped to the level bounds, giving continuous motion.
2	Checkpoints. Death returned the player to the start of the level, discarding all progress made within it.	Checkpoint tiles record the player position and confirm it with an on-screen message; a death restores the player at the last checkpoint.
3	Key bindings. Only one small set of keys was accepted, and the layout could not be changed to suit the player.	Movement, jump, fire and jetpack accept WASD, the arrow keys, Space, J or Ctrl and Shift or K at the same time, matched on physical key codes.

Sl. No.	Limitation in the Original Game	Solution Implemented
4	Visual clarity. Coarse low-resolution pixel art made hazards and platform edges difficult to read on modern displays.	The world is drawn as vector shapes at the device pixel ratio, with shaded panels and a palette in which pink always harms and mint always helps.
5	Movement physics. Jumps followed a fixed arc and the simulation was tied to machine speed, which made control imprecise.	A fixed timestep of one hundred and twentieth of a second with interpolated rendering, variable jump height, coyote time and a jump buffer.
6	Learning curve. The game offered no instruction, so the controls and the objective had to be discovered by repeated failure.	The first level is flat and safe and carries hints that appear as the player walks into them; later levels introduce the gun and the jetpack in the same way.
7	Instant death. A single contact with any hazard ended the life immediately, with no margin for error.	The player has three hearts, with knockback and a short period of flickering invulnerability after each hit; only an empty heart bar causes a respawn.
8	Saved progress. Progress existed only within a session and was lost as soon as the game was closed.	Unlocked levels, the chosen character, banked gems and the last checkpoint are written to localStorage and validated field by field when read back.
9	Player choice. A single fixed character offered no variation in movement or difficulty.	Three characters are selectable and interchangeable mid-run, each trading speed, jump height and hearts against one another.

7. System Design

The application follows a client-side-only architecture. A single Next.js route renders the React interface, which owns one canvas element and one instance of the game engine. The engine runs the simulation and all drawing, and publishes a snapshot of its state on every frame; the React components that make up the menus and the heads-up display consume that snapshot and never modify the simulation directly. Persistence is handled by a small store backed by the browser's local storage, to which the menu subscribes. There is no server logic, no database and no network request at runtime, so the built application can be served as static files.

The modules that make up the game layer, located under the src/game directory, are described below.

Module	Responsibility
theme.ts	Single source of every colour, tile dimension and tuning constant used by the game.

Module	Responsibility
levels.ts	The ten hand-authored levels, stored as character grids, together with the tutorial hint zones.
input.ts	Keyboard layer that maps physical key codes to game actions and tracks held keys and one-shot presses.
physics.ts	Player and enemy motion, gravity, jump behaviour and axis-by-axis collision against the tile grid.
render.ts	All canvas drawing routines for the world, characters, pickups, hazards and effects.
engine.ts	The game loop and state machine: level loading, pickups, combat, camera, checkpoints and snapshots for the interface.
save.ts	Reading, validating and writing the localStorage save, exposed as a small subscribable store.

8. Implementation Highlights

8.1 Fixed-Timestep Physics Loop

The simulation advances in fixed steps of one hundred and twentieth of a second. Each animation frame adds the elapsed wall-clock time to an accumulator, which is then drained one fixed step at a time, and the fraction left over is passed to the drawing routine so that positions are interpolated between the previous and the current step. The elapsed time consumed in a single frame is capped, which prevents a long pause, such as a switch to another browser tab, from producing a burst of catch-up steps. The result is that movement, gravity and collision behave identically on a sixty hertz display and on a high refresh rate one. Within this loop the player receives variable jump height, a coyote time that still permits a jump shortly after leaving a ledge, and a jump buffer that remembers a jump pressed shortly before landing.

8.2 Smooth Camera

The camera holds a horizontal position that is advanced towards a target on every step by an exponential interpolation, expressed so that the rate of approach does not depend on the length of the step. The target is the player position offset by a look-ahead distance in the direction of travel, which reveals more of the level ahead of the player than behind. The resulting position is clamped to the bounds of the level so that the view never scrolls past the first or the last column, and it is interpolated once more at draw time using the leftover accumulator fraction.

On a respawn the camera is snapped directly to its target so that the player is never shown a sweep across the level.

8.3 Checkpoint and Save System

Checkpoint tiles placed within the levels are collected in the same pass as gems and keys. Activating one records the player position, displays a confirmation message and writes the checkpoint, together with the current level, into the save. When the heart bar empties, the player is restored at that position with a full heart bar rather than at the start of the level. The save itself holds the number of unlocked levels, the chosen character, the banked gems and the last checkpoint under a single versioned key in local storage. Every field is rebuilt individually when the save is read, and any value that is missing, of the wrong type or out of range falls back to its default, so a corrupted or hand-edited entry cannot prevent the game from starting.

9. Results

The remake was completed and tested in current desktop browsers. All ten levels are playable from the main menu, either from the beginning of a new run or from the last checkpoint of a previous one. The simulation holds a steady frame rate, the camera follows the player without visible snapping, and the heads-up display reflects hearts, gems, ammunition and fuel accurately throughout play. Progress survives a reload of the page, and each of the nine limitations identified in the original game is addressed by a working feature of the remake. Selected screens of the completed application are shown below.

9.1 Screenshots

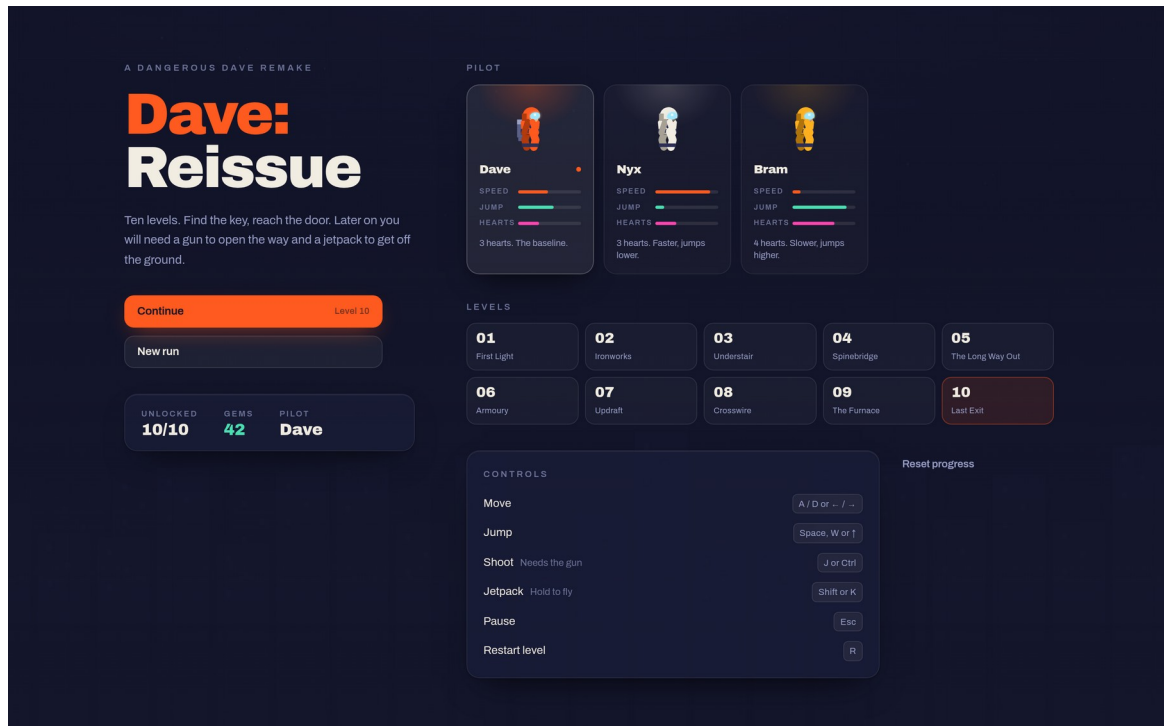


Fig. 1: Main Menu

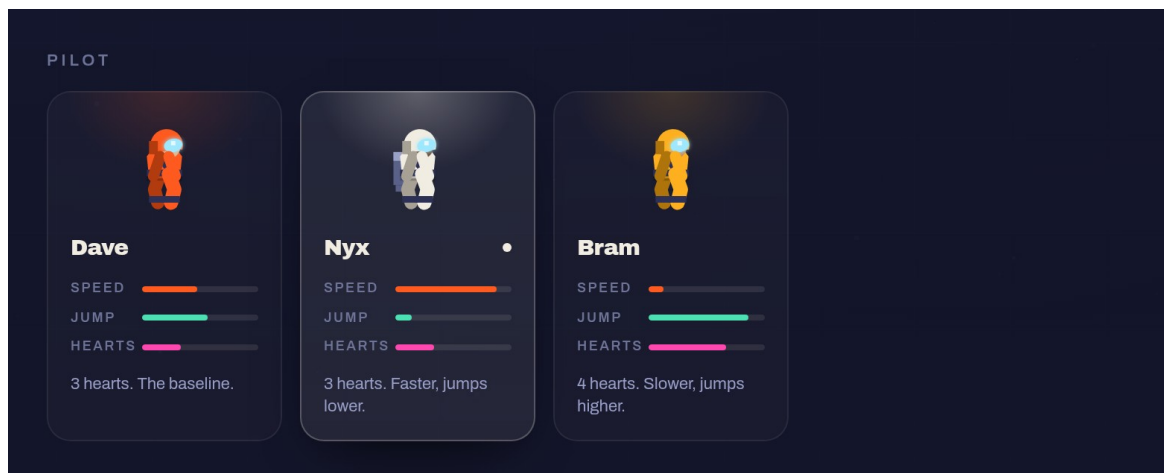


Fig. 2: Character Selection

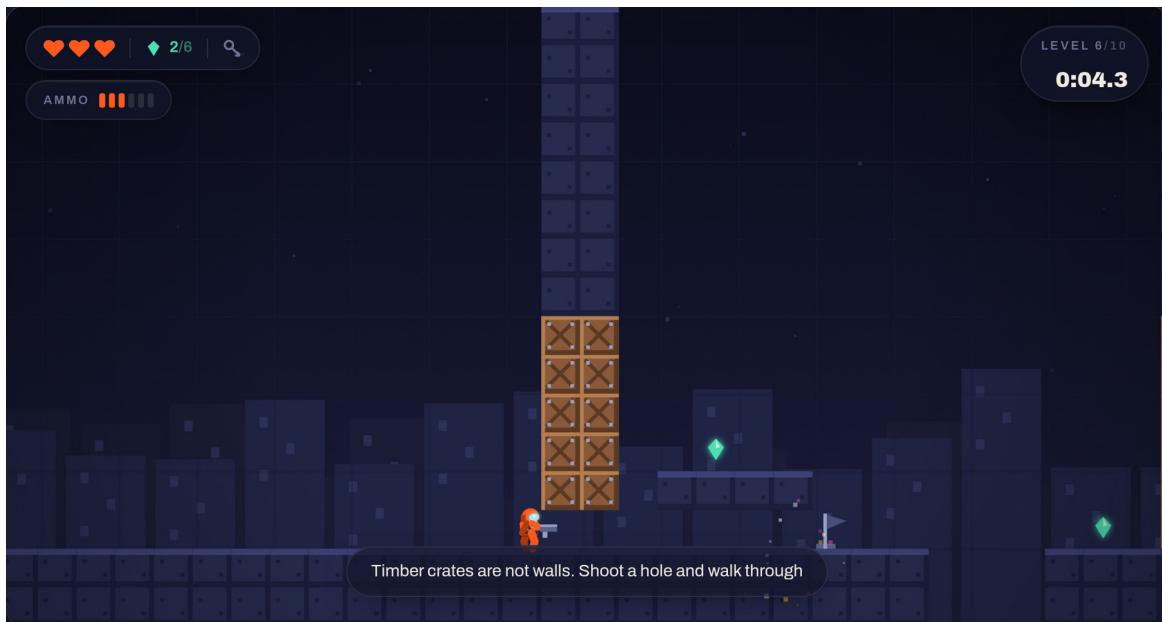


Fig. 3: Gameplay with Heads-Up Display

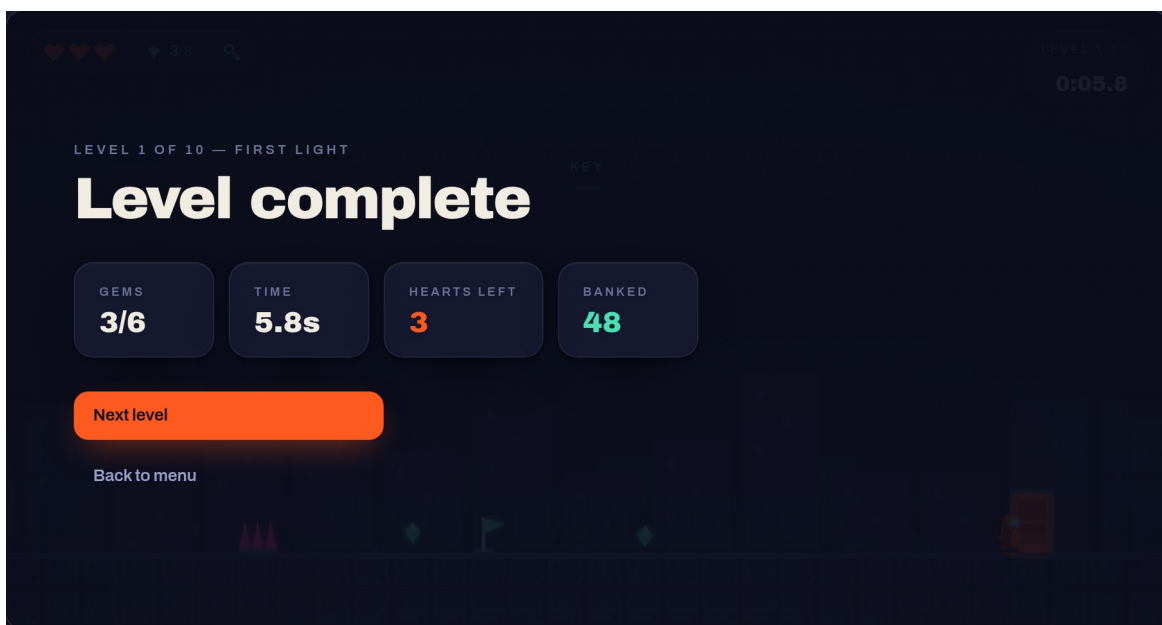


Fig. 4: Level Complete Screen

10. Conclusion

The project demonstrates that the appeal of an early platform game can be retained while its interface is rebuilt to modern standards. By treating the limitations of the original as design problems rather than as period detail, the remake replaces snapping scroll, instant death, fixed controls and lost progress with a smooth camera, a heart-based health system, simultaneous key bindings and a validated persistent save. The client-side architecture keeps the application

small and deployable as static files, and the separation between the simulation and the interface components keeps each of them straightforward to modify.

11. Future Scope

- Touch and gamepad input, so that the game can be played on tablets and with a controller.
- A level editor that writes the same tile-character format used by the existing levels.
- Sound effects and background music, with an accessible control for muting them.
- An optional online leaderboard recording completion times for each level.

12. References

[1] J. Romero and J. Hall, Dangerous Dave, Softdisk Publishing, 1988.

[2] Next.js Documentation, Vercel Inc. Available: <https://nextjs.org/docs>

[3] Canvas API, MDN Web Docs, Mozilla. Available:
https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

[4] Project repository, Dave: Reissue. Available: <https://github.com/hegdeujwal/dave-reissue>